

Air Quality Forecasting

1 Complete Coding

Task 1: Air Quality Forecasting — Complete Jupyter/Google Colab Code

Dataset: UCI Air Quality Dataset

Target: one-hour-ahead `CO(GT)`.

This notebook contains the complete coding pipeline for Parts A–F: data understanding, preprocessing, EDA, feature engineering, chronological modelling, evaluation, error analysis, leakage discussion, findings, limitations and future work.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

DATA_PATH = "/mnt/data/AirQualityUCI_submission.csv"

# For Google Colab, if the file is not at DATA_PATH, upload it:
# from google.colab import files
# uploaded = files.upload()
# DATA_PATH = next(iter(uploaded))

df_raw = pd.read_csv(DATA_PATH, sep=None, engine="python")
print("Dataset shape:", df_raw.shape)
df_raw.head()
```

Part A — Data understanding and preprocessing

```
df = df_raw.copy()
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("\nData types:")
print(df.dtypes)
print("\nMissing values encoded as -200:")
print((df == -200).sum().sort_values(ascending=False))
print("\nDate range:", df["Date"].min(), "to", df["Date"].max())

df["Datetime"] = pd.to_datetime(
    df["Date"].astype(str) + " " + df["Time"].astype(str), errors="coerce"
)
df = df.sort_values("Datetime").reset_index(drop=True)

numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
df[numeric_cols] = df[numeric_cols].replace(-200, np.nan)

# Causal imputation: forward fill only, so future observations are not used.
df[numeric_cols] = df[numeric_cols].ffill()

print("\nRemaining missing values:")
print(df.isna().sum().sort_values(ascending=False).head(10))
```

Missing-value approach

The UCI dataset uses `-200` as a missing-value code. It is converted to `NaN`. Because this is a forecasting problem, the main pipeline uses **forward fill only**, avoiding future-based interpolation. Rows that remain unusable after lag/rolling feature construction are dropped before modelling.

Part B — Exploratory Data Analysis

```
fig, ax = plt.subplots(figsize=(12,5))
ax.plot(df["Datetime"], df["CO(GT)"], linewidth=0.8)
ax.set_title("CO(GT) over Time"); ax.set_xlabel("Time"); ax.set_ylabel("CO concentration")
ax.grid(alpha=0.25); plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(8,5))
ax.hist(df["CO(GT)"].dropna(), bins=40)
ax.set_title("Distribution of CO(GT)"); ax.set_xlabel("CO concentration"); ax.set_ylabel("Frequency")
plt.tight_layout(); plt.show()

corr_cols=["CO(GT)", "NOx(GT)", "NO2(GT)", "C6H6(GT)", "T", "RH", "AH"]
corr=df[corr_cols].corr()
print(corr["CO(GT)"].sort_values(ascending=False))

fig, ax=plt.subplots(figsize=(9,7))
im=ax.imshow(corr, aspect="auto")
ax.set_xticks(range(len(corr_cols)),corr_cols,rotation=70,ha="right")
```

```

ax.set_yticks(range(len(corr_cols)),corr_cols)
ax.set_title("Correlation Matrix"); plt.colorbar(im,ax=ax); plt.tight_layout(); plt.show()

df["hour"]=df["Datetime"].dt.hour
hourly=df.groupby("hour")["CO(GT)"].mean()
fig, ax=plt.subplots(figsize=(9,4))
ax.plot(hourly.index,hourly.values,marker="o")
ax.set_title("Average CO(GT) by Hour"); ax.set_xlabel("Hour"); ax.set_ylabel("Mean CO")
ax.grid(alpha=0.25); plt.tight_layout(); plt.show()

```

Part C — Feature engineering

```

df["hour"]=df["Datetime"].dt.hour
df["day_of_week"]=df["Datetime"].dt.dayofweek
df["month"]=df["Datetime"].dt.month
df["is_weekend"]=(df["day_of_week"]>=5).astype(int)

# One-hour-ahead target.
df["target_next_CO"]=df["CO(GT)"].shift(-1)

lag_cols=["CO(GT)", "NOx(GT)", "NO2(GT)", "C6H6(GT)", "T", "RH", "AH"]
for c in lag_cols:
    df[f"{c}_lag1"]=df[c].shift(1)
    df[f"{c}_lag3"]=df[c].shift(3)
    df[f"{c}_lag24"]=df[c].shift(24)

# Shift first, then roll: only past values enter the rolling feature.
for c in ["CO(GT)", "NOx(GT)", "NO2(GT)", "C6H6(GT)"]:
    df[f"{c}_roll3"]=df[c].shift(1).rolling(3).mean()
    df[f"{c}_roll24"]=df[c].shift(1).rolling(24).mean()

feature_cols=[c for c in df.columns if c not in ["Date", "Time", "Datetime", "target_next_CO"]]
model_df=df.dropna(subset=feature_cols+["target_next_CO"]).copy()
print("Usable modelling rows:", len(model_df))
print("Number of features:", len(feature_cols))
print(feature_cols)

```

Part D — Prediction

```

split_idx=int(len(model_df)*0.80)
train=model_df.iloc[:split_idx].copy()
test=model_df.iloc[split_idx:].copy()

X_train=train[feature_cols]; y_train=train["target_next_CO"]
X_test=test[feature_cols]; y_test=test["target_next_CO"]

print("Training rows:", len(train), "Testing rows:", len(test))
print("Training period:", train["Datetime"].min(), "to", train["Datetime"].max())
print("Testing period:", test["Datetime"].min(), "to", test["Datetime"].max())

model=RandomForestRegressor(
    n_estimators=200, max_depth=18, min_samples_leaf=2,
    random_state=42, n_jobs=-1
)
model.fit(X_train, y_train)
pred=model.predict(X_test)

mae=mean_absolute_error(y_test, pred)
mse=mean_squared_error(y_test, pred)
rmse=np.sqrt(mse)
r2=r2_score(y_test, pred)

results=pd.DataFrame({"Metric": ["MAE", "MSE", "RMSE", "R²"], "Value": [mae, mse, rmse, r2]})
display(results.round(4))

```

Part E — Analysis and leakage control

```

pred_df=test[["Datetime", "target_next_CO"]].copy()
pred_df["Prediction"]=pred
pred_df["Error"]=pred_df["target_next_CO"]-pred_df["Prediction"]
pred_df["Absolute_Error"]=pred_df["Error"].abs()

fig,ax=plt.subplots(figsize=(12,5))
ax.plot(pred_df["Datetime"], pred_df["target_next_CO"], label="Actual", linewidth=0.9)
ax.plot(pred_df["Datetime"], pred_df["Prediction"], label="Predicted", linewidth=0.9)
ax.set_title("Actual vs Predicted Next-Hour CO"); ax.set_xlabel("Time"); ax.set_ylabel("CO")
ax.legend(); ax.grid(alpha=0.25); plt.tight_layout(); plt.show()

importance=pd.Series(model.feature_importances_, index=feature_cols).sort_values(ascending=False)
print("Top 15 features:")
print(importance.head(15))

fig,ax=plt.subplots(figsize=(9,6))
importance.head(15).sort_values().plot(kind="barh", ax=ax)
ax.set_title("Top 15 Random Forest Feature Importances"); ax.set_xlabel("Importance")
plt.tight_layout(); plt.show()
print("\nLeakage controls:")

```

```
print("1. Chronological train/test split – no random shuffling.")
print("2. Target is one hour ahead.")
print("3. Lag features use earlier observations only.")
print("4. Rolling features are shifted before rolling.")
print("5. Missing values are forward-filled only.")
```

Part F — Findings, limitations and future improvement

5–7 key findings

1. CO concentration varies substantially over time, making temporal features useful.
2. Recent pollutant history is informative for one-hour-ahead CO forecasting.
3. Hour-of-day and calendar variables capture recurring temporal patterns.
4. Other pollutant and meteorological variables provide additional predictive information.
5. Random Forest can capture nonlinear relationships among the engineered features.
6. Larger errors tend to occur around unusual/high-concentration observations.
7. Chronological evaluation provides a more realistic forecasting assessment than random splitting.

Limitations

1. Forward filling can carry an old value across a long missing interval.
2. Random Forest does not explicitly model sequential dynamics like dedicated time-series models.
3. Only one holdout period and one forecast horizon were evaluated.

Future improvement

Compare Random Forest with gradient boosting and dedicated time-series/deep-learning models, use rolling-origin cross-validation and hyperparameter tuning, and evaluate multiple forecast horizons.